DyLas: Bridging Dynamic Networks and Language Model Semantic Spaces for Generalizable Adaptive Bitrate Streaming

Anonymous submission

Abstract

Adaptive bitrate (ABR) streaming selects the bitrate of each video chunk under fluctuating network conditions to maximize Quality of Experience (QoE). Existing methods either rely on hand-crafted control rules or learn task-specific policies from numerical features, providing limited structured modeling of trajectories. Recent environment-aware approaches capture network dynamics with latent models, but they do not exploit the pretrained sequence-modeling and representation capabilities of language models. Consequently, their generalization to unseen time-varying network conditions remains limited. To address this issue, we introduce DyLas, an LLM-based ABR framework that bridges **Dy**namic networks and **La**nguage model **S**emantic spaces. DyLas leverages the LLM's sequence modeling capacity to capture transferable patterns from network states, bitrate decisions, and QoE feedback. It extracts trajectory features and aligns them with the LLM embedding space. To capture uncertainty in evolving network conditions, we propose a bandwidth-aware state encoder which summarizes recent throughput and download-time histories. The LLM then acts as the policy and predicts the next bitrate from the aligned trajectory context. DyLas achieves the highest average QoE on the in-distribution dataset, while the strongest baselines are 4.7% below DyLas on average across the out-of-distribution datasets.

Introduction

Adaptive bitrate (ABR) streaming divides a video into consecutive chunks, each encoded at multiple bitrate levels. Before downloading each chunk, the client selects a bitrate based on the current playback buffer and observed network measurements. However, the client cannot know how bandwidth will change while the selected chunk is being downloaded. The effect of bandwidth fluctuations becomes known only after the download completes, when the next state is observed. The delayed feedback makes ABR an online sequential decision problem, requiring each decision to balance immediate video quality, the risk of rebuffering and bitrate smoothness (Bentaleb et al. 2018). A high bitrate improves visual quality but can cause rebuffering when bandwidth drops, while a low bitrate reduces stall risk at the cost of quality.

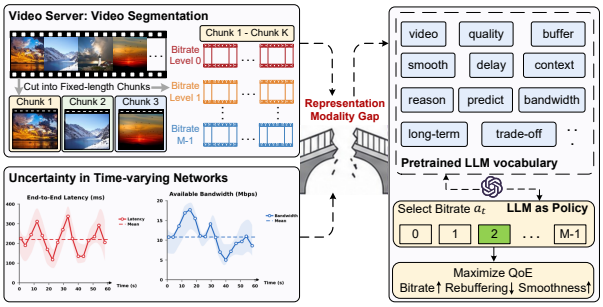


Figure 1: Overview of adaptive bitrate decision making and its representation challenge for pretrained language models. For each video chunk, the client selects one of M bitrate levels to balance video quality, rebuffering, and bitrate smoothness under time-varying bandwidth. ABR observations and decision histories are numerical trajectories, whereas pretrained language models operate on token embeddings, creating a representation gap between the two spaces.

Classical ABR methods use buffer rules, bandwidth prediction, or utility optimization (Huang et al. 2014; Spiteri, Urgaonkar, and Sitaraman 2020; Yin et al. 2015), while learning-based methods learn bitrate policies or latent environment representations from numerical network states (Mao, Netravali, and Alizadeh 2017; Huang et al. 2019; Xia et al. 2022; Huang et al. 2026). Even when they explicitly infer network uncertainty, these methods remain task-specific numerical policies. Recent work has also explored pretrained language models for network decision tasks (Wu et al. 2024; He et al. 2024). Their sequence modeling capacity and pretrained representation space are promising for ABR, where states, bitrate decisions, and QoE feedback form an evolving trajectory. However, ABR observations consist of heterogeneous numerical variables, whereas language models operate in a token-embedding space. Existing task adapters (Houlsby et al. 2019) can match input dimensions, but dimensional compatibility alone does not preserve the roles of ABR variables or provide a structured representation of changing network conditions. Figure 1 illustrates this mismatch. Thus, the key problem is **how to bridge the representation gap**

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

between numerical network state trajectories and the embedding space of pretrained language models.

To address this problem, we propose DyLas, an LLM-based framework that aligns structured ABR trajectories with the embedding space of a pretrained LLM to improve generalization across varying network conditions. DyLas contains two core modules: the Network Semantic Alignment module and Bandwidth-Aware State Encoder. The network semantic alignment module encodes heterogeneous ABR variables. Its block-masked self-attention (BMSA) submodule models the directed dependencies among variables within each state transition. The bandwidth-aware state encoder summarizes recent throughput and download-time histories and fuses the resulting network representation with the state pathway. The adapted LLM then processes the aligned trajectories and serves as the policy for predicting the next bitrate.

The main contributions of this paper are summarized as follows:

- We formulate LLM-based ABR as a trajectory representation alignment problem and propose DyLas. The network semantic alignment module jointly encodes structured ABR transitions and maps numerical state, action, and reward representations into the pretrained LLM embedding space.
- We develop a bandwidth-aware state encoder for time-varying networks. This encoder processes recent throughput and download-time histories, and augments each state with a compact representation of network dynamics before semantic alignment.
- We achieve the state-of-the-art average QoE across all the evaluated in-distribution and out-of-distribution network traces. Using DyLas' average QoE for normalization, the strongest baseline is 10.8% below DyLas on in-distribution data, and the strongest baselines are 4.7% below DyLas on average across the out-of-distribution datasets.

Related Work

Classical ABR Algorithms

Classical ABR algorithms usually make bitrate decisions through explicit control principles, including buffer-based rules, model predictive control, and Lyapunov optimization. Buffer-based methods select bitrates according to playback buffer size, avoiding direct dependence on uncertain future bandwidth estimates. BBA (Huang et al. 2014) is a representative method that maps buffer states to bitrate choices to reduce rebuffering while maintaining video quality. RobustMPC (Yin et al. 2015) formulates ABR as finite-horizon QoE optimization and plans future bitrate choices using predicted bandwidth, buffer state, and video chunk size. BOLA (Spiteri, Urgaonkar, and Sitaraman 2020) derives an online control policy from utility maximization and Lyapunov optimization, allowing bitrate selection without an explicit bandwidth predictor. These methods are lightweight and interpretable, but their decision logic is largely specified before deployment. As a result, they have limited ability to learn richer representations from historical network-state trajectories.

Learning-Based ABR Algorithms

Learning-based ABR algorithms improve bitrate adaptation by learning policies, predictors, or environment representations from data. Pensieve (Mao, Netravali, and Alizadeh 2017) trains a neural ABR policy with reinforcement learning to optimize QoE under dynamic network conditions. Oboe (Akhtar et al. 2018) auto-tunes ABR algorithms to network conditions, and Comyco (Huang et al. 2019) uses imitation learning so that a policy network can imitate expert decisions with future information. MERINA infers bandwidth-related latent conditions from historical observations (Kan et al. 2022). GENET (Xia et al. 2022) improves generalization by automatically generating training curricula for reinforcement-learning-based network policies. EAStream (Huang et al. 2026) formulates ABR as context-based meta-reinforcement learning: a recurrent variational encoder infers a probabilistic environment belief from interaction history, and an actor–critic policy conditions on this belief to adapt to unseen conditions without test-time fine-tuning. These studies demonstrate the value of modeling hidden network dynamics, but their policies remain task-specific models over numerical state features. In contrast, DyLas uses a bandwidth-specific latent representation without a state- or reward-reconstruction objective and aligns structured state-action-reward trajectories with a pretrained sequence model.

Language Models for Networking and Adaptive Streaming

Recent studies have explored different roles for language and foundation models in networking and adaptive streaming. NADA (He et al. 2024) uses LLMs to generate network algorithms, while ComTree (Jia et al. 2025) studies LLM-assisted interpretation of decision-tree ABR policies. Content-side methods such as JumpDASH (Wang et al. 2025) and PivotSketch (Zhang et al. 2026) extract video-semantic information to support streaming decisions. These methods demonstrate the potential of foundation models for algorithm generation, policy interpretation, and content-aware adaptation, but they do not directly address the representation of numerical trajectories for bitrate control. Sequence-based decision models, such as Decision Transformer (Chen et al. 2021) and Trajectory Transformer (Janner, Li, and Levine 2021), further show that ordered decision trajectories can be represented as token-like sequences. Most closely related to DyLas, NetLLM (Wu et al. 2024) adapts a frozen LLM to ABR and other networking tasks using task-specific input encoders and LoRA modules. However, its input adaptation does not explicitly preserve the directed dependencies within each transition or separately model latent network dynamics.

Method

Problem Formulation

A video contains K chunks, each encoded at M bitrate levels. Before downloading chunk t, the client observes state s_t and selects $a_t \in \mathcal{A} = \{0, \ldots, M-1\}$. The per-chunk QoE reward r_t is defined as follows:

$$r_t = U(a_t) - \alpha d_t - \beta |U(a_t) - U(a_{t-1})|. \quad (1)$$

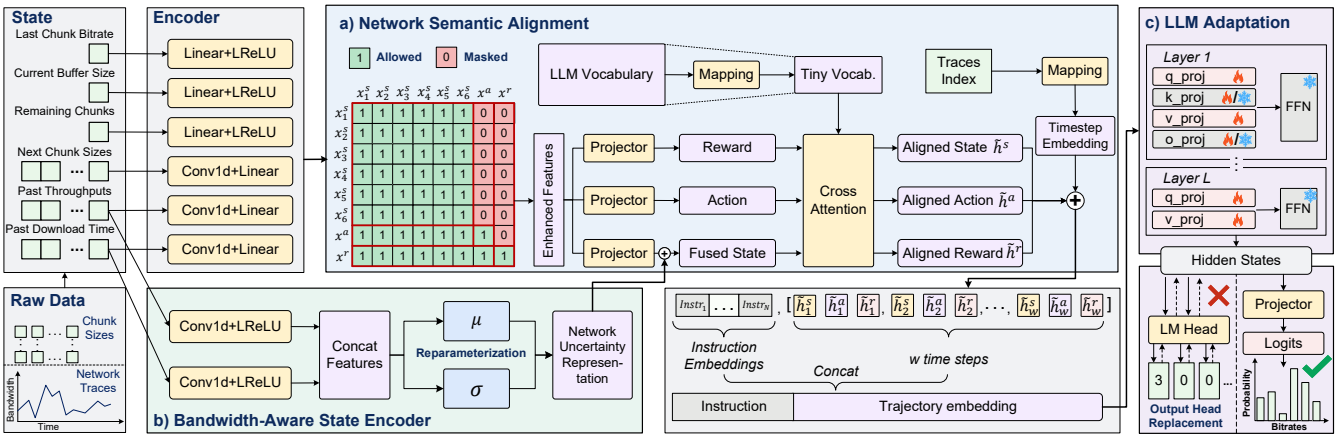


Figure 2: Overall architecture of DyLas. a) The network semantic alignment module encodes ABR trajectories (i.e., heterogeneous state, action, and reward variables), applies block-masked self-attention to preserve the directed dependency structure within each completed transition, and aligns the resulting representations with a compact basis derived from the pretrained LLM vocabulary. b) The bandwidth-aware state encoder summarizes recent throughput and download-time histories as a stochastic latent representation and fuses it with the state pathway before alignment. c) The aligned trajectory sequence is processed by a low-rank adapted Llama backbone, and a task-specific bitrate head predicts the next bitrate from the hidden state.

where $U(a_t)$ is bitrate utility, d_t is rebuffering duration, and α and β weight rebuffering and bitrate variation.

Specifically, the state s_t follows the commonly used Pensieve (Mao, Netravali, and Alizadeh 2017) state representation and contains two distinct types of input observations: (i) scalar state (i.e., last chunk bitrate, current buffer size); (ii) history-based and bitrate-candidate state (i.e., remaining chunks, next chunk sizes, past throughput, and past download time). DyLas generates each bitrate decision based on the state s_t and a sliding window containing no more than w completed state–action–reward transitions, which is defined as follows:

$$\mathcal{H}_t^{(w)} = \left[(s_i, a_i, r_i)\right]_{i=\max(1,t-w)}^{t-1}. \tag{2}$$

Given $\mathcal{H}_t^{(w)}$ and the current state s_t, DyLas models the conditional bitrate action distribution as follows:

$$\pi_\theta(a_t = a \mid \mathcal{H}_t^{(w)}, s_t), \quad a \in \mathcal{A}. \tag{3}$$

The sliding window bounds LLM inference cost while retaining recent network dynamics and QoE feedback. Thus, the policy π_θ maximizes expected session QoE:

$$\max_{\pi_\theta} \mathbb{E}_{\pi_\theta}\left[\sum_{t=1}^{K} r_t\right]. \tag{4}$$

To parameterize the policy in Eq. 3, DyLas constructs role-specific state, action, and reward representations. It then augments the state pathway with latent network dynamics, and aligns the resulting trajectory sequence with the pretrained LLM embedding space. Figure 2 summarizes the full architecture.

Network Semantic Alignment

For each timestep, scalar state groups are mapped by linear layers, while history-based and bitrate-candidate groups are encoded by 1D-CNN layers. These feature encoders produce six state-feature tokens $x_{i,1}^s, \ldots, x_{i,6}^s$, together with an action token x_i^a and a reward token x_i^r for each completed transition. We collect them as the ordered role-token sequence.

$$X_i = [x_{i,1}^s, \ldots, x_{i,6}^s, x_i^a, x_i^r] \tag{5}$$

Structure-Aware Feature Extraction. From a reinforcement learning perspective, state, action, and reward are not exchangeable variables: the action is conditioned on the observed state, while the reward is obtained after action execution. Therefore, DyLas applies block-masked self-attention (BMSA) to X_i within each timestep to preserve this asymmetric dependency structure. The block mask follows the causal order of an ABR transition. Each state-feature query attends only to the six state features; the action query attends to the state features and itself; and the reward query attends to the state features, action, and itself. We formalize the output of BMSA as follows:

$$\widehat{X}_i = \mathrm{BMSA}(X_i; \mathcal{M}) = [\hat{x}_{i,1}^s, \ldots, \hat{x}_{i,6}^s, \hat{x}_i^a, \hat{x}_i^r], \tag{6}$$

where $\mathcal{M}$ is the role-aware block mask. The six contextualized state tokens are combined by learned normalized weights ω_j and projected to the state-encoder representation e_i^s. Distinct linear projection layers are independently adopted to generate output embeddings for the action token and reward token, respectively:

$$e_i^s = W_s\left(\sum_{j=1}^{6} \omega_j \hat{x}_{i,j}^s\right), \quad e_i^a = W_a \hat{x}_i^a, \quad e_i^r = W_r \hat{x}_i^r. \tag{7}$$

Thus, e_i^s depends only on s_i, e_i^a depends on (s_i, a_i), and e_i^r depends on (s_i, a_i, r_i). For the current timestep, only the six state-feature tokens are available, and the same state block and weighted fusion produce e_t^s without using the unknown

current action or reward. Before semantic alignment, the bandwidth-aware state encoder augments the state pathway according to Eq. 14, yielding $\bar{e}_i^s$ for completed transitions and $\bar{e}_t^s$ for the current decision; the action and reward pathways retain e_i^a and e_i^r.

Alignment stage. Although BMSA retains the directed role structure of each trajectory, the resulting representations remain in an ABR-specific numerical feature space. Therefore, we align them with the pretrained LLM embedding space before constructing the trajectory sequence. Let $E \in \mathbb{R}^{|\mathcal{V}|\times d}$ denote the LLM token embedding matrix. Directly using the full vocabulary as alignment memory is expensive because $|\mathcal{V}|$ is large, and many token embeddings are unrelated to ABR decisions. DyLas projects E along the vocabulary dimension into a compact token-basis matrix $\tilde{E} \in \mathbb{R}^{N_v\times d}$, where $N_v \ll |\mathcal{V}|$. For a representation e from any pathway, instantiated as $\bar{e}_i^s$ for the state pathway, e_i^a for the action pathway, and e_i^r for the reward pathway, the aligned embedding is produced by shared cross-attention:

$$\tilde{h} = \mathrm{CrossAttn}(W_q e, W_k\tilde{E}, W_v\tilde{E}), \tag{8}$$

By sharing this cross-modal alignment interface across the state, action, and reward pathways, we obtain semantically aligned state, action and reward, denoted as $\tilde{h}_i^s$, $\tilde{h}_i^a$, and $\tilde{h}_i^r$, respectively. A learnable timestep embedding is added to each aligned representation to preserve temporal order.

DyLas prepends a fixed natural-language task prompt that provides a shared task context across trajectories. The LLM tokenizer converts the prompt into token IDs, and the corresponding rows of E form the prompt-embedding sequence $P = [p_1, \ldots, p_{n_p}]$. The complete aligned decision sequence is

$$u_t = [P, (\tilde{h}_i^s, \tilde{h}_i^a, \tilde{h}_i^r)_{i=\max(1,t-w)}^{t-1}, \tilde{h}_t^s], \tag{9}$$

The exact prompt is fixed across all training and evaluation trajectories and is provided in the supplementary material.

Moreover, the aligned sequence is processed by the pretrained Llama 3.2-1B base model. The original backbone parameters are kept frozen, and LoRA adapters are applied to the query and value projections of each self-attention layer. We extract the hidden state at the position corresponding to the current-state representation $\tilde{h}_t^s$ and replace the native language-modeling head with a linear bitrate-prediction head g_ϕ. Let θ denote all trainable LoRA and task-specific parameters. The resulting policy is as follows:

$$\pi_\theta(a_t \mid \mathcal{H}_t^{(w)}, s_t) = \mathrm{softmax}\Big(g_\phi([\mathrm{LLM}(u_t)]_{\mathrm{pos}(\tilde{h}_t^s)})\Big). \tag{10}$$

where $[\mathrm{LLM}(u_t)]_{\mathrm{pos}(\tilde{h}_t^s)}$ denotes the LLM hidden state at the current-state position. The policy therefore predicts the next bitrate from the aligned historical transitions and the current state, rather than generating tokens from the original language-model vocabulary.

Bandwidth-Aware State Encoder

Bandwidth variation directly affects download time and rebuffering. Although the conventional state representation method already includes finite histories of throughput and download time, independently encoding these raw observations may not explicitly expose the latent network condition jointly reflected by them. Moreover, the two histories are short, noisy, and coupled. DyLas therefore summarizes them into a compact stochastic representation and injects it into the state pathway before semantic alignment. For each numerical ABR state s_t, we extract a two-channel network-history trajectory from past throughput and past download time:

$$x_t^b = [x_t^{\mathrm{tp}}, x_t^{\mathrm{dt}}] \in \mathbb{R}^{L\times 2}, \tag{11}$$

where x_t^{tp} and x_t^{dt} denote the throughput and download-time histories, respectively. The encoder defines a diagonal-Gaussian posterior over the latent bandwidth representation:

$$q_\psi(z_t^b \mid x_t^b) = \mathcal{N}\big(\mu_\psi(x_t^b), \mathrm{diag}\big(\sigma_\psi^2(x_t^b)\big)\big). \tag{12}$$

We sample from this posterior using the reparameterization:

$$z_t^b = \mu_\psi(x_t^b) + \epsilon \odot \sigma_\psi(x_t^b), \quad \epsilon \sim \mathcal{N}(0, I). \tag{13}$$

where $\odot$ denotes element-wise multiplication. The latent variable z_t^b provides a compact stochastic representation of recent network-condition measurements. This encoder has no reconstruction target; it is trained only through the bitrate-prediction objective and the KL regularizer defined below. During inference, DyLas uses the mean $\mu_\psi(x_t^b)$ to obtain a deterministic representation.

DyLas projects z_t^b to the state-representation dimension and fuses it with the state-encoder representation:

$$\bar{e}_t^s = e_t^s + W_b z_t^b. \tag{14}$$

The fused state representation $\bar{e}_t^s$ is returned to network semantic alignment to produce $\tilde{h}_t^s$.

Training and Inference

Directly optimizing Eq. 4 requires repeated interaction with the streaming environment. Instead, DyLas approximates a high-QoE policy through behavior cloning from offline trajectories generated by an oracle dynamic-programming expert and strong learned ABR policies. Historical states, actions, and QoE rewards form the input context, while the recorded next action serves as the supervision target. For each timestep, the model receives the aligned sequence in Eq. 9 and predicts the recorded trajectory action with supervised classification as follows:

$$\mathcal{L}_{\mathrm{cls}} = -\frac{1}{N}\sum_{n=1}^{N} \log \pi_\theta(a_n \mid \mathcal{H}_n^{(w)}, s_n), \tag{15}$$

Trace set	Scenario	Avg. BW (Mbps)
FCC	In-Dist. U.S. broadband	1.37 ± 0.78
Norway3G	OOD 3G transit	2.04 ± 0.41
Oboe	OOD video sessions	2.67 ± 1.35
HSR	OOD high-mobility wireless	7.64 ± 4.74

Table 1: Network trace datasets used in evaluation. In-dist. and OOD are defined relative to the shared trace-source exposure of DyLas and the trainable baselines. The average bandwidth reports the mean and standard deviation of per-trace average throughput.

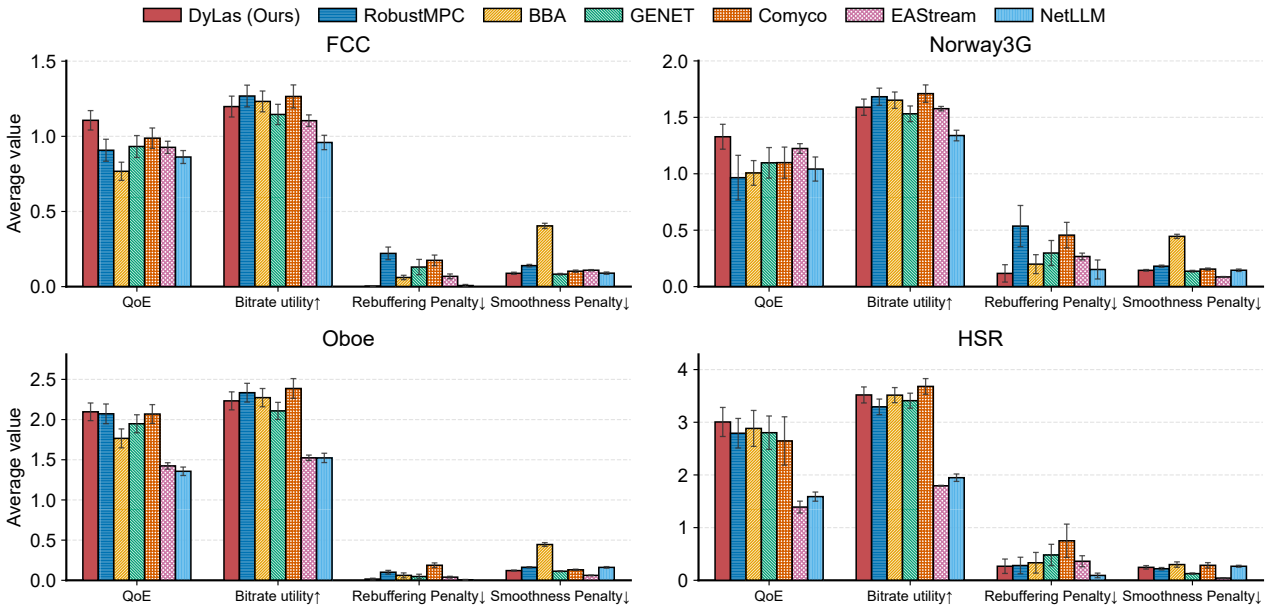


Figure 3: Decomposition of average QoE on FCC, Norway3G, Oboe, and HSR. For each method, the bars report the average session QoE, bitrate utility, weighted rebuffering penalty and weighted smoothness penalty. Error bars denote the standard deviation across network traces. Higher values are preferred for QoE and bitrate utility, whereas lower values are preferred for the two penalty components.

where N is the number of supervised decision steps in a minibatch.

The bandwidth-aware state encoder is regularized by the KL divergence between the latent posterior and a standard Gaussian prior:

$$\mathcal{L}_{\mathrm{bw}} = \frac{1}{N}\sum_{n=1}^{N} D_{\mathrm{KL}}\big(q_\psi(z_n^b \mid x_n^b)\,\|\,\mathcal{N}(0,I)\big)\,. \tag{16}$$

The final training objective is $\mathcal{L} = \mathcal{L}_{\mathrm{cls}} + \lambda_{\mathrm{bw}}\mathcal{L}_{\mathrm{bw}}$, and λ_{bw} controls the strength of the KL regularization.

During offline training, we jointly optimize the LoRA (Hu et al. 2022) adapters, ABR-specific feature encoders, BMSA, bandwidth-aware state encoder, semantic alignment module, timestep embeddings, and bitrate-prediction head, while keeping the original Llama backbone frozen. At inference time, DyLas constructs $\mathcal{H}_t^{(w)}$ from completed chunks, predicts a bitrate distribution with Eq. 10, and samples a_t from π_θ. After the chunk is downloaded, the observed state transition and QoE reward are appended to the decision context. The procedure then repeats until the video session ends.

Experiments

Experiments use Python 3.10.19, PyTorch 2.9.1+cu128 (CUDA 12.8 runtime), Transformers 4.57.1, and PEFT 0.18.0 on one NVIDIA GeForce RTX 5090 D GPU (32 GB). Each 60-epoch DyLas training run with Llama 3.2-1B in FP32 takes approximately 4.01 hours. We measure the one-step decision latency after 10 warm-up runs and report the mean and standard deviation over 50 repetitions. DyLas requires 49.87 ± 1.65 ms per decision, whereas NetLLM requires 45.37 ± 3.03 ms under the same GPU setting. DyLas' average latency accounts for only 1.25% of the 4-second video chunk duration used in our experiments, indicating that it remains compatible with the evaluated online ABR decision timescale. Further details are provided in the supplementary material.

Experimental Setup

Network Trace Datasets. We conduct evaluations on four real-world trace datasets, as summarized in Table 1. The FCC (Federal Communications Commission 2016) dataset serves as the in-distribution training trace source. In contrast, Norway3G (Riiser et al. 2013), Oboe (Akhtar et al. 2018), and HSR (Meng et al. 2019) are held out from all trainable methods and used for cross-dataset out-of-distribution (OOD) evaluation. Here, generalization denotes transfer to unseen trace sources under a fixed simulator, video content, bitrate ladder, and QoE objective. We use the bitrate set $\{300, 750, 1200, 1850, 2850, 4300\}$ kbps. Each video chunk has a duration of 4 seconds, the maximum buffer capacity is 60 seconds, and each video sequence contains 49 chunks. Therefore, $M = 6$ and $K = 49$ in all experiments.

Baselines. We compare DyLas with BBA (Huang et al. 2014), RobustMPC (Yin et al. 2015), Comyco (Huang et al. 2019), GENET (Xia et al. 2022), EAStream (Huang et al. 2026), and NetLLM (Wu et al. 2024) in a common simulator. BBA and RobustMPC require no training, whereas all trainable baselines are trained exclusively on FCC traces. To ensure a controlled comparison between the two LLM-based methods, NetLLM and DyLas use the same Llama 3.2-

Method	FCC	Norway3G	Oboe	HSR
RMPC	0.91 ± 0.73	0.97 ± 1.27	2.07 ± 1.22	2.79 ± 1.64
BBA	0.77 ± 0.60	1.01 ± 0.70	1.77 ± 1.18	2.88 ± 1.99
GENET	0.93 ± 0.73	1.10 ± 0.87	1.95 ± 1.12	2.80 ± 1.85
Comyco	0.99 ± 0.68	1.10 ± 0.88	2.07 ± 1.17	2.65 ± 2.67
EAStream	0.93 ± 0.56	1.22 ± 0.59	1.42 ± 0.55	1.39 ± 1.56
NetLLM	0.86 ± 0.43	1.04 ± 0.68	1.36 ± 0.53	1.59 ± 0.51
DyLas	$\mathbf{1.11 \pm 0.65}$	$\mathbf{1.33 \pm 0.71}$	$\mathbf{2.10 \pm 1.10}$	$\mathbf{3.01 \pm 1.61}$

Table 2: Average trace QoE of DyLas and the compared ABR baselines. RMPC denotes RobustMPC. The values report the mean and standard deviation across traces. Best results are shown in bold.

1B backbone and the same offline pool of 600 trajectories. Specifically, the pool contains 300 trajectories generated by a dynamic-programming expert and 150 trajectories each generated by GENET and Comyco. The dynamic-programming expert has access to the complete bandwidth trace and video-chunk sequence and selects bitrate actions to maximize cumulative session QoE. Further details on trajectory construction, baseline implementation, and optimization are provided in the supplementary material.

Evaluation Metric. We adopt the QoE reward defined in Eq. 1 as our evaluation metric. Following the linear QoE paradigm widely used in prior ABR evaluation, $U(a_t)$ (in Mbps) is the bitrate selected by action a_t, with the parameters set to $\alpha = 4.3$ and $\beta = 1$. To further analyze the source of QoE discrepancies, we evaluate the average bitrate-utility component $U(a_t)$, the weighted rebuffering penalty αd_t, and the weighted smoothness penalty $\beta|U(a_t) - U(a_{t-1})|$. All methods are evaluated under the same network traces, video chunks, bitrate ladder, and QoE parameters.

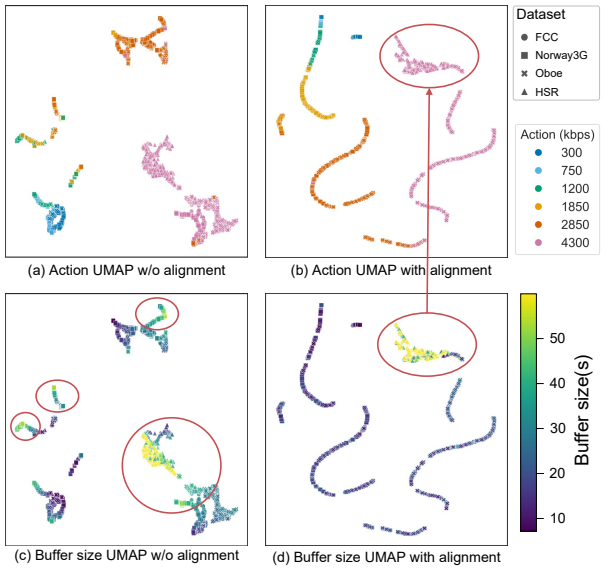


Figure 4: UMAP visualizations of state representations produced by independent linear projection and network semantic alignment. The left and right columns use the same state samples before and after semantic alignment, respectively. Samples are colored by the selected bitrate action in the top row and by playback-buffer size in the bottom row; marker shapes indicate the trace dataset. Bitrate actions are measured in kbps, and buffer size is measured in seconds. Red ellipses and arrows highlight representative regions used for qualitative comparison.

Performance and Generalization

Table 2 reports average QoE. For each dataset, we normalize the strongest baseline's shortfall by DyLas' average QoE; Supplementary Figure 1 provides the corresponding session-level distributions. As shown in Table 2, DyLas achieves the highest average QoE on all four datasets. The strongest baselines are 10.8%, 8.3%, 1.4%, and 4.3% below DyLas on FCC, Norway3G, Oboe, and HSR, respectively, yielding an average shortfall of 4.7% across the OOD datasets and 6.2% across all four. The three OOD sets cover 3G transit, heterogeneous commercial sessions, and high-mobility wireless traces. Notably, HSR exhibits a markedly higher average bandwidth and greater bandwidth variability than the FCC training traces, representing a pronounced shift in both mobility and bandwidth. Despite this shift, DyLas achieves the highest average QoE on HSR, supporting its generalization beyond the low-bandwidth conditions observed during training. DyLas also maintains consistent gains on Norway3G and Oboe without retraining, although the margin on Oboe is modest. On FCC, DyLas further outperforms GENET and Comyco, which generate half of its offline trajectory pool. Under the matched trajectory pool and Llama 3.2-1B backbone, DyLas consistently surpasses NetLLM across all evaluated datasets.

Figure 3 shows that the QoE gains are not explained by bitrate utility alone. EAStream is the strongest baseline on Norway3G, where its bitrate utility is close to DyLas but its larger rebuffering penalty lowers QoE. On Oboe and HSR, its lower QoE mainly reflects lower bitrate utility.

Ablation Study

Table 3 evaluates four variants that remove BMSA or the bandwidth-aware state encoder, replace semantic alignment with independent linear projections, or replace the pretrained backbone with a frozen randomly initialized model. Figure 5 reports their session-level QoE distributions, while Supplementary Figure 2 provides the QoE component breakdowns.

BMSA and semantic alignment matter most on Norway3G: removing them reduces QoE by 25% and 28%, respectively, while the corresponding FCC reductions are 14% and 16%. These results support preserving within-transition dependencies and using vocabulary alignment rather than dimensional projection alone. Removing the bandwidth-aware state encoder has modest effects on FCC and Norway3G but reduces QoE by 27% on HSR and 14% on Oboe, largely through lower bitrate utility. Pretrained initialization has the largest overall effect: the randomly initialized variant produces negative QoE on FCC and Norway3G and remains below DyLas on HSR and Oboe, demonstrating the importance of pretrained representations under the fixed-backbone protocol.

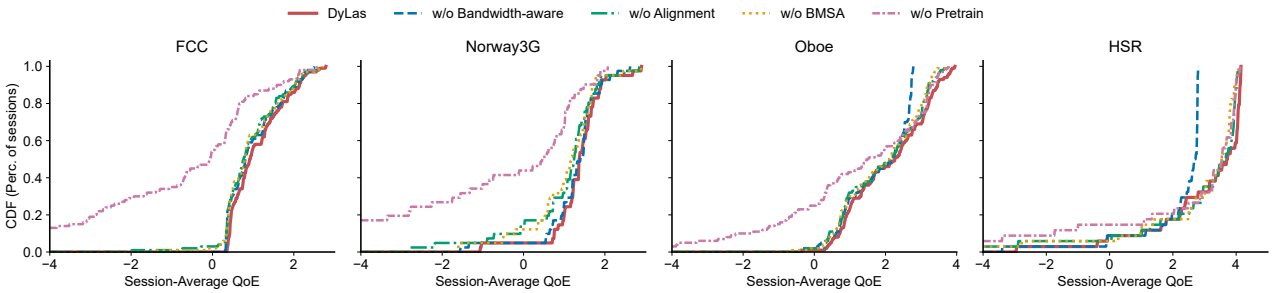


Figure 5: Empirical cumulative distribution functions of session-average QoE for DyLas and its ablation variants on FCC, Norway3G, Oboe, and HSR. For each session, QoE is averaged over all downloaded chunks. A rightward shift indicates higher QoE over a larger fraction of evaluated sessions.

The CDFs in Figure 5 support these average results. Removing BMSA or semantic alignment shifts the lower and middle Norway3G percentiles leftward, removing the bandwidth-aware state encoder causes larger shifts on HSR and Oboe, and the randomly initialized variant shows the broadest degradation.

Representation Analysis

Figure 4 compares state representations produced by independent linear projection and network semantic alignment. Without semantic alignment, the projected representations exhibit fragmented local clusters, where samples with different bitrate actions and buffer states are interleaved, indicating that dimensional projection alone cannot organize heterogeneous ABR variables according to decision-related factors. After semantic alignment, the representations form smoother trajectories with clearer action- and buffer-related structures while maintaining overlap among different trace datasets. This suggests that the proposed alignment learns decision-relevant organization beyond trace-specific distribution matching, enabling the LLM-compatible representation to preserve useful ABR semantics.

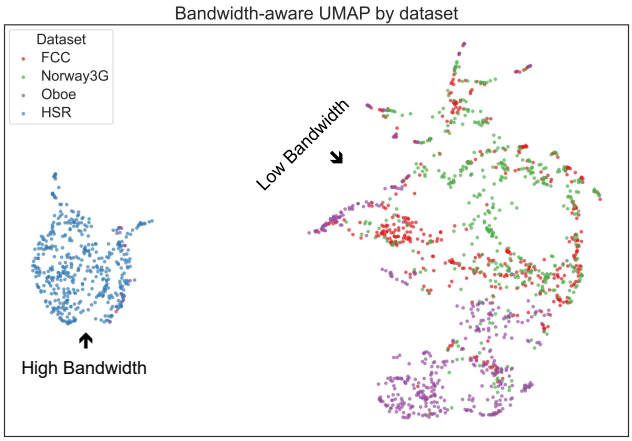


Figure 6: UMAP visualization of the bandwidth-aware representation z_t^b. Colors denote trace datasets, and the annotations mark representative low- and high-bandwidth regions.

Variant	FCC	Norway3G	Oboe	HSR
DyLas	$\mathbf{1.11 \pm 0.65}$	$\mathbf{1.33 \pm 0.71}$	$\mathbf{2.10 \pm 1.10}$	$\mathbf{3.01 \pm 1.61}$
w/o BMSA	0.96 ± 0.70	1.00 ± 0.90	1.91 ± 1.07	2.77 ± 1.94
w/o Align.	0.93 ± 0.72	0.96 ± 1.08	1.94 ± 1.10	2.82 ± 2.03
w/o BWA	1.01 ± 0.64	1.24 ± 0.76	1.80 ± 0.91	2.20 ± 1.23
w/o Pretrain	-1.13 ± 2.97	-0.94 ± 3.03	1.04 ± 2.46	2.41 ± 2.64

Table 3: Average trace QoE of DyLas and its ablation variants. Align. and BWA denote semantic alignment and the bandwidth-aware state encoder, respectively. The complete DyLas model is shown in bold.

Figure 6 shows a bandwidth-correlated organization of z_t^b: relatively low- and high-bandwidth states occupy different regions, while samples from different trace sources remain partially overlapped. This pattern suggests that the encoder captures bandwidth-related network variations without reducing the representation to trace source identity. Together, the visualizations support decision-related organization after semantic alignment and sensitivity to network dynamics.

Conclusion

We proposed DyLas, a trajectory representation alignment framework for ABR under dynamic network conditions. DyLas connects numerical state–action–reward histories to a pretrained LLM through the network semantic alignment module and the bandwidth-aware state encoder. Across the evaluated real-world in-distribution and out-of-distribution traces, DyLas attains the highest average QoE among the compared rule-based, optimization-based, learning-based, and LLM-based methods. Component and ablation analyses support the complementary contributions of the proposed components and pretrained initialization. Future work will investigate efficient deployment and broader networked decision tasks.

References

Akhtar, Z.; Nam, Y. S.; Govindan, R.; Rao, S.; Chen, J.; Katz-Bassett, E.; Ribeiro, B.; Zhan, J.; and Zhang, H. 2018. Oboe: Auto-tuning video ABR algorithms to network conditions. In *ACM SIGCOMM*, 44–58.

Bentaleb, A.; Taani, B.; Begen, A. C.; Timmerer, C.; and Zimmermann, R. 2018. A survey on bitrate adaptation schemes for streaming media over HTTP. *IEEE Communications Surveys & Tutorials*, 21(1): 562–585.

Chen, L.; Lu, K.; Rajeswaran, A.; Lee, K.; Grover, A.; Laskin, M.; Abbeel, P.; Srinivas, A.; and Mordatch, I. 2021. Decision Transformer: Reinforcement Learning via Sequence Modeling. *Advances in Neural Information Processing Systems*, 34: 15084–15097.

Federal Communications Commission. 2016. Raw Data - Measuring Broadband America 2016.

He, Z.; Gottipati, A.; Qiu, L.; Luo, X.; Xu, K.; Yang, Y.; and Yan, F. Y. 2024. Designing network algorithms via large language models. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, 205–212.

Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; De Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*, 2790–2799. PMLR.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, volume 1, 3.

Huang, T.; Zhou, C.; Zhang, R.-X.; Wu, C.; Yao, X.; and Sun, L. 2019. Comyco: Quality-aware adaptive video streaming via imitation learning. In *Proceedings of the 27th ACM international conference on multimedia*, 429–437.

Huang, T.-Y.; Johari, R.; McKeown, N.; Trunnell, M.; and Watson, M. 2014. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. In *Proceedings of the 2014 ACM conference on SIGCOMM*, 187–198.

Huang, Z.; Xiao, W.; Chen, M.; Liu, Z.; Chen, M.; Vasilakos, A. V.; Farouk, A.; and Song, H. H. 2026. EAStream: An Environment-Aware Adaptive Bitrate Algorithm for Reliable Video Streaming Services. *IEEE Transactions on Services Computing*.

Janner, M.; Li, Q.; and Levine, S. 2021. Offline Reinforcement Learning as One Big Sequence Modeling Problem. *Advances in Neural Information Processing Systems*, 34: 1273–1286.

Jia, L.; Li, C.; Yuan, Z.; Chen, J.; Huang, T.; Liu, J.; and Sun, L. 2025. Beyond interpretability: Exploring the comprehensibility of adaptive video streaming through large language models. In *Proceedings of the 33rd ACM International Conference on Multimedia*, 12035–12044.

Kan, N.; Jiang, Y.; Li, C.; Dai, W.; Zou, J.; and Xiong, H. 2022. Improving generalization for neural adaptive video streaming via meta reinforcement learning. In *Proceedings of the 30th ACM international conference on multimedia*, 3006–3016.

Mao, H.; Netravali, R.; and Alizadeh, M. 2017. Neural adaptive video streaming with pensieve. In *Proceedings of the conference of the ACM special interest group on data communication*, 197–210.

Meng, Z.; Chen, J.; Guo, Y.; Sun, C.; Hu, H.; and Xu, M. 2019. PiTree: Practical implementation of ABR algorithms using decision trees. In *ACM MM*, 2431–2439.

Riiser, H.; Vigmostad, P.; Griwodz, C.; and Halvorsen, P. 2013. Commute path bandwidth traces from 3G networks: Analysis and applications. In *ACM MMSys*, 114–118.

Spiteri, K.; Urgaonkar, R.; and Sitaraman, R. K. 2020. BOLA: Near-optimal bitrate adaptation for online videos. *IEEE/ACM transactions on networking*, 28(4): 1698–1711.

Wang, H.; Zhou, T.; Li, Q.; Jiang, Y.; and Muntean, G.-M. 2025. JumpDASH: LLM-Based Content Perception for Intelligent Jumping DASH in Mobile Adaptive Video Streaming. *IEEE Transactions on Networking*.

Wu, D.; Wang, X.; Qiao, Y.; Wang, Z.; Jiang, J.; Cui, S.; and Wang, F. 2024. NetLLM: Adapting Large Language Models for Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 661–678.

Xia, Z.; Zhou, Y.; Yan, F. Y.; and Jiang, J. 2022. Genet: Automatic curriculum generation for learning adaptation in networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, 397–413.

Yin, X.; Jindal, A.; Sekar, V.; and Sinopoli, B. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM conference on special interest group on data communication*, 325–338.

Zhang, S.; Li, Y.; Zhang, C.; Fan, X.; Hu, X.; and Duan, H. 2026. PivotSketch: Control-Ready Semantic Ranking for Adaptive Video Streaming. In *Proceedings of the 36th Workshop on Network and Operating System Support for Digital Audio and Video*, 141–147.